

llm-memory: A Local-First Canonical Memory Layer for AI Agents

Separating Conclusions from Evidence in Agent Memory Systems

Rafael Salema Marques
github.com/salemarsm/llm-memory

White Paper, Version 0.1.0-alpha
May 2026

Abstract

Modern AI agents require persistent memory that survives across sessions, projects, and model providers. Current solutions conflate three distinct concerns—raw conversation history, retrieved document fragments, and durable canonical knowledge—into a single retrieval surface, typically a vector database populated by language-model extractions. This conflation has measurable costs: opaque provenance, weak auditability, and structural dependence on the same model that produced the memories in the first place. This paper presents *llm-memory*, a local-first canonical memory layer for AI agents. The system separates events, evidence, and canonical conclusions into distinct primitives backed by SQLite as the single source of truth, exposes them through the Model Context Protocol (MCP) for transparent agent integration, and treats embeddings as replaceable indexes rather than primary storage. We describe the data model, retrieval pipeline, supersession lifecycle, and audit trail; analyze the system against representative alternatives (Mem0, Letta, Zep, Cognee); and discuss limitations and a roadmap toward governance, hybrid retrieval, and evidence-aware RAG. The reference implementation is open source under the MIT license and currently distributed as version 0.1.0-alpha.

Keywords: agent memory, local-first software, retrieval, BM25, SQLite, FTS5, Model Context Protocol, RAG, auditability, supersession.

1. Introduction

Large Language Models (LLMs) are stateless by design. Each call to a model is independent: the model has no intrinsic memory of prior interactions, prior decisions, or prior corrections. When LLMs are embedded into agents—software systems that issue multi-turn requests, manipulate external state, and persist across sessions—this statelessness becomes a structural problem. Agents must reconstruct context on every turn, either by replaying conversation history, retrieving from external stores, or relying on the user to repeat themselves.

The dominant industry response has been to introduce a memory subsystem alongside the

agent: a service that captures conversational fragments, embeds them, stores them in a vector database, and retrieves them by semantic similarity at query time. Variations of this pattern underlie commercial offerings such as Mem0, Zep, Cogniee, and the proprietary memory features of consumer assistants. These systems are pragmatic and often effective, but they share a set of architectural assumptions that, in our view, conflate fundamentally different kinds of information.

This paper argues that durable agent memory should be designed around a different primitive: the *canonical memory record*. A canonical memory is a structured, auditable conclusion—not a fragment of dialogue, not a chunk of a document, not a vector. Vectors, document chunks, and raw conversational events have their place; they are evidence, not conclusion. Conflating them collapses provenance, complicates audit, and binds the memory layer to the LLM that produced it.

We present *llm-memory*, a working implementation of this thesis. The system is local-first, single-binary (Go), backed by SQLite with FTS5 full-text search, and integrates with agents through the Model Context Protocol. It is open source and currently at version 0.1.0-alpha, with deliberate scope: a stable canonical core, transparent retrieval, and an honest roadmap for governance, hybrid ranking, and RAG bridging.

1.1 Contributions

- A canonical memory data model that separates events, evidence, conclusions, and projection layers, with explicit provenance, confidence, scope, and supersession lifecycle.
- A retrieval pipeline that uses BM25 over an FTS5 index for candidate generation, applies subject-scope fallbacks for conversational queries that miss textual matches, enforces confidence thresholds, and emits auditable retrieval events.
- A feedback loop based on context identifiers and structured feedback events, designed to close the loop between retrieval decisions and downstream usefulness without coupling to any specific tokenizer or evaluation protocol.
- A native Model Context Protocol server that exposes memory operations as transparent agent tools (`memory_context`, `memory_suggest`, `memory_remember`, `memory_search`), with documented integrations for Claude Code, Codex-style CLIs, and OpenClaw.
- An open-source reference implementation in approximately 2,300 lines of Go, with no CGO dependencies, distributable as a single binary across Linux, macOS, and Windows.

2. Background and Motivation

2.1 The conflation problem

Most production agent memory systems treat the following as substitutable forms of “memory”:

- Raw conversational history—the literal sequence of user and assistant turns.
- Retrieved document chunks—segments of source documents surfaced via semantic search.
- Canonical durable knowledge—stable preferences, decisions, facts that should govern future agent behavior.

These categories have different lifecycles, different provenance models, and different appropriate retrieval strategies. A user preference (“prefers concise technical answers”) should outlive any specific conversation in which it was expressed. A document chunk is meaningful only as evidence for a claim, with the document itself as primary source. A conversational turn is mostly transient—useful for short-term coherence, rarely useful as durable knowledge.

Conflating them has measurable consequences. When all three are stored as embeddings in the same vector index, the agent cannot distinguish “I should follow this rule” from “this is something I once said” from “this is a citation I might quote.” When all three are extracted via the same LLM pass at write time, provenance collapses to “the model thought this was important.” When the user wishes to inspect, correct, or audit the memory, they encounter an opaque vector store rather than a structured record.

2.2 Three illustrative limitations of vector-first memory

We identify three concrete limitations that motivated the design of *llm-memory*.

Audit opacity. When a vector-first memory system “remembers” an incorrect or stale fact, the user cannot easily answer questions such as: which conversation produced this memory? when was it created? was it superseded? what was its confidence? Vector stores typically expose embeddings and payloads, but the curation logic that decided to write the memory in the first place is not preserved as a first-class object.

Operational weight. Self-hosting common alternatives requires PostgreSQL with pgvector, a separate vector database (Qdrant, Weaviate, Milvus), or a managed service. For developers who want to run a memory layer alongside agents on a workstation, a Raspberry Pi, or a containerized environment, this footprint is disproportionate to the value delivered.

Vendor coupling. Memory features built into specific assistants (proprietary memory in commercial chat products) are not portable. A user who switches between Claude Code, Codex-like CLIs, and locally hosted models cannot carry their durable preferences and project decisions across these tools.

2.3 The local-first principle

Local-first software, as articulated by Kleppmann *et al.* [2], emphasizes user agency, longevity of data, and resilience to network or vendor failure. Memory for AI agents is a natural fit for this principle: the data being stored represents the user’s preferences, their projects’ decisions, and their working knowledge. It is precisely the class of data that should not be hostage to a

third-party service. *llm-memory* is designed so that the canonical store—a single SQLite file—is fully owned, inspectable, and portable by the user.

3. Design Principles

The system is governed by five principles, in priority order.

3.1 Memory is conclusion; RAG is evidence

A canonical memory is a curated assertion: “the user prefers Go code examples”; “this project uses SQLite as the local store.” A document chunk, by contrast, answers a different question: “where did this claim come from?” The two should not share a representation. *llm-memory* stores canonical memories as structured records and stores supporting evidence as separate document and chunk tables, with optional links between them.

3.2 SQLite is the source of truth

The single SQLite database file is authoritative. All other representations—FTS5 indexes, embedding references, projected context strings—are derived. This invariant has practical consequences: the system can be inspected with standard `sqlite3` tooling, backed up by file copy, version-controlled if the user wishes, and migrated by SQL.

3.3 Embeddings are optional indexes

Vector embeddings are useful as one signal in retrieval, but they are not canonical memory. The schema reflects this by storing embeddings as references (`embedding_refs`) rather than as columns of vector type. A future change of embedding provider or model should not invalidate the memory store.

3.4 The LLM is a client, not a database

Many systems delegate memory curation to the LLM itself: the model summarizes conversations, extracts facts, and writes them back to memory. In *llm-memory*, the LLM may suggest candidates—via the `memory_suggest` tool—but the durable write is a deliberate operation, not an automatic side effect of inference. This decouples the memory layer from the model’s reliability and biases.

3.5 Auditability is a feature, not an afterthought

Every meaningful operation—memory creation, supersession, deletion, context build, context feedback—is recorded as an append-only event. The schema enforces non-destructive supersession (old memories carry a `superseded_by` reference rather than being overwritten). This design makes the system suitable for environments where forensic-grade traceability is required, while remaining lightweight enough for casual personal use.

4. System Architecture

4.1 Component overview

The reference implementation is composed of four binaries and a shared core library:

- **memserver**—HTTP server with OpenAPI-documented endpoints and an embedded local web GUI.
- **memmcp**—Model Context Protocol server speaking JSON-RPC over stdio.
- **memctl**—Command-line client for direct interaction with the HTTP API.
- **llm-memory**—Helper CLI for project initialization, diagnostics, and MCP configuration generation.

All four binaries link the same **memory** package, which contains the data model, store operations, retrieval, and context projection. The system has no required external dependencies beyond Go 1.22 and uses modernnc.org/sqlite to avoid CGO.

4.2 Data model

The canonical memory record is the central entity:

```
type Memory struct {
    ID          string
    Type        MemoryType // preference, fact, decision, task, note,
              relationship
    Subject     string      // who or what the memory is about
    Content     string
    Source      Source     // provenance: kind + ref
    Scope       Scope     // global, project, session, private
    Confidence  float64    // [0, 1], CHECK-enforced
    CreatedAt   time.Time
    UpdatedAt   time.Time
    ValidFrom   *time.Time
    ValidUntil  *time.Time
    SupersedesID *string
    SupersededBy *string
    Tags        []string
    EmbeddingRefs map[string]string
}
```

Each field encodes a design decision. **Type** and **Scope** make the memory’s role and visibility explicit. **Source** preserves provenance: a memory without provenance is, in this model, weak memory. **Confidence** is bounded and validated at the schema level. **SupersedesID** and **SupersededBy** implement non-destructive replacement. **EmbeddingRefs** is a string map rather than a vector column, enforcing the “embeddings are external indexes” principle.

Two auxiliary tables—**Documents** and **Chunks**—provide the evidence layer for RAG-style use cases, with FTS5 indexes for full-text search. A separate **context_feedback** table stores feedback on retrieval decisions. A **schema_migrations** table records applied migrations. All timestamps are stored as RFC3339Nano strings for portability.

4.3 Storage layer

The store opens SQLite with WAL journaling enabled and a single open connection (`SetMaxOpenConns(1)`) to avoid `SQLITE_BUSY` errors under concurrent writes from a single binary. Migrations are idempotent and version-tagged. IDs are UUIDs prefixed by entity type (`mem_`, `evt_`, `doc_`, `chk_`, `ctx_`, `cfb_`), avoiding the time-based collisions of an earlier design and providing predictable inspection.

Tags are stored both as a JSON column on the memory record (for round-trip API fidelity) and in a normalized **memory_tags** table (for performant filtering). This duplication is intentional: the JSON column preserves the user’s view of the record, while the normalized table supports indexed queries such as “all memories tagged *X* within scope *Y*.”

4.4 Retrieval pipeline

Retrieval is the operation by which a prompt or query is converted into a compact, prompt-ready set of canonical memories. The pipeline has five logical stages:

1. **Query normalization.** Conversational queries are tokenized using a Unicode-aware regular expression (`\pL\pN_`), lowercased, filtered against a small Portuguese/English stopword list, and recombined as an FTS5 OR-of-prefixes expression. Short tokens are retained when they appear to be technical (containing digits or uppercase letters), preserving terms such as “AI”, “Go”, “v8”, “k8s”.
2. **Candidate generation.** The normalized query is matched against the **memories_fts** virtual table. Results are ordered by BM25 score (ascending, since SQLite’s BM25 implementation returns lower-is-better), with ties broken by confidence and recency.
3. **Subject fallback.** If FTS yields no candidates and a subject is specified, the system falls back to retrieving recent high-confidence memories for that subject. This handles the conversational case where the user prompt shares no surface tokens with the durable memories that should govern the response.
4. **Filtering.** Superseded memories are excluded by SQL. Memories with confidence below 0.5 are excluded from automatic context injection, regardless of textual relevance. Scope and type filters are applied as supplied.
5. **Budget packing.** A greedy fill respects a token budget (default 1,200, capped at 8,000), estimated using a deterministic rune-based approximation. Memories that exceed the remaining budget cause the response to be marked truncated.

The result is wrapped in a `ContextResponse` that includes a `context_id`, the projected text, the selected memory records, and budget metadata. A `context.built` event is appended to the audit log when at least one memory is selected.

4.5 Feedback loop

The system instruments retrieval to permit closed-loop evaluation without committing to any single ranking algorithm. Each `ContextResponse` carries a `context_id`. Clients (or the user, via the GUI) may submit feedback through `POST /api/feedback` referencing this id, indicating whether the retrieved set was useful and which specific memories were ultimately incorporated. Feedback is stored in the `context_feedback` table and emitted as a `context.feedback` event.

The feedback signal is not currently used to alter ranking. It is collected so that future versions can perform retrospective evaluation, A/B testing of ranking strategies, and supervised tuning of confidence thresholds and decay parameters.

4.6 Integration surfaces

Three integration surfaces share the same canonical model:

- **HTTP API.** JSON over HTTP with OpenAPI documentation. Endpoints exist under both `/api/*` (legacy) and `/api/v1/*` (versioned), enabling forward compatibility. A `/healthz` endpoint supports orchestration. The HTTP server enforces `ReadHeaderTimeout`, `ReadTimeout`, `WriteTimeout`, and `IdleTimeout` to resist slow-loris-style abuse even on loopback.
- **Model Context Protocol.** A JSON-RPC server over stdio, conformant to MCP protocol version 2024-11-05 [1]. Four tools are exposed: `memory_context` (token-budgeted retrieval), `memory_suggest` (heuristic candidate generation), `memory_remember` (durable write), and `memory_search` (raw search). The buffer is configured for payloads up to 10 MiB.
- **CLI.** `memctl` provides scriptable access to all primary operations and is the recommended client for non-MCP automation.

5. Implementation Notes

5.1 Scale and shape of the codebase

The implementation is deliberately small. As of v0.1.0-alpha, the codebase is approximately 2,300 lines of Go across the core memory package, the HTTP server, the MCP server, and the four CLI binaries. A reader can audit the entire system in a single sitting. This is itself a feature: the smaller the surface, the easier it is to reason about correctness, performance, and security.

5.2 Schema migrations

Migrations are stored in code as a list of idempotent SQL statements applied at store open. A `schema_migrations` table records the applied version. `ALTER TABLE` statements that have already been applied are tolerated via duplicate-column detection. This pattern is sufficient for

the current scope and avoids the operational burden of a migration framework.

5.3 Concurrency and durability

WAL journaling is enabled to permit concurrent reads alongside writes. The single open connection serializes writes within a process, which is appropriate for the single-user local-first deployment model. Multi-writer scenarios are explicitly out of scope for v0.x.

5.4 Testing posture

Tests focus on invariants rather than implementation details. Representative cases include: that BM25-ranked preferences win over recent weakly-related tasks; that subject-scope fallback recovers durable memories when FTS misses; that low-confidence memories never enter automatic context; that empty contexts do not pollute the audit log; and that technical short tokens (e.g., “AI”, “k8s”) survive query normalization. These tests are intended to remain valid as the underlying ranking strategy evolves.

6. Comparison with Related Systems

A useful frame for situating *llm-memory* is a four-quadrant matrix along two axes: storage canonicity (vector-first vs structured-first) and deployment posture (hosted vs local-first). Most active systems occupy three of the four quadrants. The local-first, structured-first quadrant is comparatively underpopulated, and that is where *llm-memory* situates itself.

System	Primary store	Local-first	Audit model
Mem0 [7]	Vector DB + LLM extractions	Hosted-first	Inferred (LLM extracted)
Letta (MemGPT) [6]	Agent-internal memory tiers	Hosted or self-hosted	Coupled to agent runtime
Zep [8]	PostgreSQL + pgvector + graph	Self-hostable, heavy-weight	Conversational fact extraction
Cognee [9]	Knowledge graph + vector	Multi-backend	LLM-driven graph construction
Engram	SQLite + FTS5 + MCP	Local-first	Session/observation oriented
<i>llm-memory</i> [10]	SQLite + FTS5 + events	Local-first	Canonical, append-only events, supersession

Table 1: Comparison of agent memory systems along storage and audit dimensions.

6.1 Engram

Engram is the closest known peer in technical posture: Go, SQLite, FTS5, MCP. The projects differ primarily in lens: Engram emphasizes coding-agent observations and session memory; *llm-memory* emphasizes a canonical lifecycle (events, evidence, conclusions, supersession, projection) intended to support coding agents but to generalize beyond them. The systems can be complementary, with Engram serving as a coding-agent surface and *llm-memory* serving as a

longer-term canonical store.

6.2 Mem0, Zep, Cognition

These systems are mature and offer features (managed hosting, conversational extraction pipelines, multi-tenant isolation) that *llm-memory* does not. They are appropriate choices when those features are needed. They do not, however, prioritize local-first deployment, single-binary distribution, or canonical separation of evidence from conclusion in the way *llm-memory* does. For users for whom auditability and ownership outweigh convenience, the tradeoff favors a structured-first, local-first design.

6.3 Vendor-coupled assistant memory

Memory features built into commercial chat assistants are convenient but not portable across tools, models, or vendors. A user who works with multiple agents (a coding CLI, a generic chat assistant, a local model) will find that none of these built-in memories share state. A neutral memory layer—accessed via MCP—makes durable preferences and decisions portable across this fragmented landscape.

7. Limitations and Threats to Validity

We summarize the principal limitations of the current implementation honestly. These are reflected in the public roadmap and backlog.

- **Heuristic suggestion engine.** `memory_suggest` currently uses a keyword-based cascade with hardcoded Portuguese and English triggers. This is sufficient for proof of concept and for a “human in the loop” workflow where every candidate requires confirmation, but it is not the intended long-term architecture. The roadmap calls for a **Suggester** interface with pluggable implementations, including LLM-backed suggesters.
- **No authentication.** The HTTP API has no authentication in v0.1.0-alpha. Deployment is intended to be loopback-only. API tokens are scheduled for v0.2 as an exit criterion before any non-loopback exposure.
- **Token estimation.** `EstimateTokens` uses a deterministic rune-count heuristic ($\lfloor n/4 \rfloor + 1$). This is acceptable for budgeting but diverges from real BPE tokenizers, particularly for non-Latin scripts. A pluggable tokenizer interface is on the roadmap.
- **Single-tenant assumption.** The store assumes a single trusted user. There is no row-level security, no per-user encryption, and no scoped credential model. Multi-tenant deployments are not supported and not currently a goal.
- **No vector retrieval.** BM25 + subject fallback covers many cases but cannot handle queries whose semantic relationship to relevant memories is purely paraphrastic. Hybrid retrieval combining BM25 with optional embeddings is on the roadmap.

- **No contradiction detection.** When two memories make incompatible claims, the system does not surface the conflict at write or retrieval time. Governance primitives—conflict relations, supersession recommendations, confidence reinforcement—are scheduled for v0.4.
- **Hard delete only.** `Forget` is a destructive SQL `DELETE`. Soft-delete semantics, including delete-with-tombstone for audit, are scheduled for v0.4. Backup of the SQLite file is currently the recommended safety practice.

8. Roadmap

The project follows a versioned roadmap with explicit exit criteria for each release.

- **v0.1 — Local core.** Canonical store, FTS5 retrieval, MCP server, GUI. Mostly implemented; remaining items are CI hardening, GoReleaser packaging, version-stamped binaries, and a benchmark harness.
- **v0.2 — Local safety and installability.** API tokens, auth-aware MCP propagation, install scripts, structured logging, smoke tests. *Exit criterion:* HTTP API is not unauthenticated by default beyond loopback.
- **v0.3 — Agent integration quality.** Tested integrations for Claude Code, Codex-style CLIs, OpenClaw. Progressive-disclosure retrieval (`memory_get`, `memory_timeline`, `memory_evidence`). One-command setup with dry-run.
- **v0.4 — Governance and memory quality.** Contradiction detection, supersession recommendation, conflict/reinforcement relations, soft delete, audit GUI, sensitive-data detection, approval queue.
- **v0.5 — Retrieval quality.** Hybrid ranking (BM25 + optional embeddings + recency decay parameterized by type), MMR-based diversification, subject-hierarchy aware retrieval, confidence-aware scoring.

9. Conclusion

AI agents need memory, but not all memory is alike. Conflating raw conversation, retrieved evidence, and durable canonical knowledge into a single vector index simplifies implementation at the cost of audit, ownership, and clarity. *llm-memory* is an argument, in code, that a different design is feasible: a small, local-first, structured-first system in which canonical memories are first-class, evidence is preserved separately, embeddings are replaceable, and the LLM is a client rather than a database.

The v0.1.0-alpha implementation demonstrates that this design is practical: a single Go binary, a single SQLite file, an MCP-native integration story, and a retrieval pipeline whose behavior is testable, auditable, and inspectable. The roadmap acknowledges what remains: authentication, governance, hybrid retrieval, and a richer suggestion engine. The project is open source under

the MIT license at github.com/salemarism/llm-memory and welcomes contributions, scrutiny, and disagreement.

References

- [1] Anthropic. Model Context Protocol Specification, 2024-11-05 revision. <https://modelcontextprotocol.io>.
- [2] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan. Local-first software: You own your data, in spite of the cloud. In *Proc. Onward!*, 2019.
- [3] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [4] D. R. Hipp. SQLite FTS5 extension documentation. <https://www.sqlite.org/fts5.html>.
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [6] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [7] Mem0 documentation. <https://docs.mem0.ai>.
- [8] Zep documentation. <https://www.getzep.com/docs>.
- [9] Cognee documentation. <https://docs.cognee.ai>.
- [10] *llm-memory* source repository. <https://github.com/salemarism/llm-memory>.

A. Project Status Snapshot (May 2026)

- Version: v0.1.0-alpha.1
- License: MIT
- Language: Go 1.22
- Dependencies: modernnc.org/sqlite, [google/uuid](https://google.com/uuid) (no CGO)
- Approximate code size: ~2,300 lines of Go
- Test coverage: invariant-focused unit and integration tests
- Distribution: source build today; GoReleaser packaging in progress